■ AI-Powered Resume Analyzer

Complete Project Notes

Madhunika Danam · Project 4 · Built in 14 Days

Live Frontend	https://madhu-resume-analyzer.streamlit.app
Live Backend API	https://ml-journey.onrender.com
API Docs	https://ml-journey.onrender.com/docs

1. Project Overview

What I built: A full-stack AI application that analyzes a resume against a job description and returns:

- A **match score** (0–100)
- A list of **missing keywords**
- One **actionable suggestion** to improve the resume

Full Tech Stack

Layer	Tool	Purpose
Backend	FastAPI	REST API framework — handles routes, validation, JSON
AI	Anthropic Claude API (claude-sonnet-4-6)	Analyzes resume vs job description
Prompt Engineering	Custom prompt	Forces structured JSON output
Frontend	Streamlit	Web UI — file upload, display results
PDF Parsing	pdfplumber	Extracts text from uploaded PDF resumes
Data Validation	Pydantic	Validates incoming request data automatically
Server	Uvicorn	ASGI server that runs FastAPI
Security	python-dotenv	Loads API key from .env file safely
Backend Deploy	Render	Cloud platform running FastAPI 24/7
Frontend Deploy	Streamlit Cloud	Cloud platform running Streamlit 24/7
Version Control	Git + GitHub	Daily commits, code history

2. Complete Definitions — Every Term Explained

Python & General

Python

A high-level programming language known for its simple, readable syntax. Used for data analysis, machine learning, web development, and AI integration. The entire project is written in Python.

Virtual Environment (venv)

An isolated Python environment for each project. Keeps packages separated so one project's dependencies don't conflict with another's. Created with python -m venv venv, activated with venv\Scripts\Activate.ps1 on Windows.

pip

Python's package installer. Used to install libraries like FastAPI, Streamlit, and pdfplumber. Command: pip install package_name.

requirements.txt

A text file listing all Python packages a project needs. Anyone can recreate the exact environment by running pip install -r requirements.txt. Generated with pip freeze > requirements.txt.

f-string

A Python string formatting method that allows variables to be inserted directly into text using {} placeholders. Example: f"Hello, {name}" → "Hello, Madhu". The f prefix before the quotes activates this feature.

Dictionary

A Python data structure that stores key-value pairs in curly braces. Example: {"message": "hi", "score": 72}. Each value is accessed by its key, not position. Very similar to JSON format.

if/elif/else

Python's conditional logic structure. if runs when a condition is True, elif checks the next condition if the first was False, else runs when all conditions above were False.

Function

A reusable block of code defined with def that performs a specific task. Functions take inputs (parameters) and return outputs. Example: def say_hi(): return {"message": "hi"}.

Decorator

A special Python syntax using @ that wraps a function with extra behavior. In FastAPI, @app.get("/") registers a URL route — it connects a URL address to the function below it.

Class

A Python template/blueprint for creating objects with specific attributes. In this project, class ResumeRequest(BaseModel): defines the shape of incoming data.

import

Python keyword that loads a package or module into your file. from fastapi import FastAPI means go into the fastapi package and bring in the FastAPI tool specifically.

json.loads()

Python function that converts a JSON-formatted string into a Python dictionary. loads = load string. Used to parse Claude's text response into usable Python data.

os.getenv()

Python function that reads environment variables from the system. Used to safely access the API key stored in the .env file without hardcoding it in source code.

FastAPI

FastAPI

A modern Python web framework for building REST APIs. Extremely fast, automatically validates data using Pydantic, auto-generates interactive documentation at /docs, and converts Python dictionaries to JSON automatically.

REST API

A design pattern for web APIs where different URLs represent different resources, and HTTP methods (GET, POST, PUT, DELETE) represent actions on those resources. The most common API design pattern.

HTTP

HyperText Transfer Protocol — the communication language that browsers and servers use to talk to each other over the internet. Every website visit and API call uses HTTP.

GET Request

An HTTP method for retrieving/reading data from a server. No data is sent in the request body. Used when you just want to fetch something. Example: GET / to check if the API is alive.

POST Request

An HTTP method for sending/submitting data to a server. Data travels in the request body. Used when you want to send information for the server to process. Example: POST /analyze to submit a resume.

Route (Path Operation)

A specific URL endpoint that triggers a function when visited with the right HTTP method. @app.get("/") registers a GET route, @app.post("/analyze") registers a POST route.

Path Parameter

A variable part of a URL that FastAPI captures automatically. In /hello/{name}, the {name} part captures whatever text is typed there and passes it to the function.

Request Body

The data payload sent with a POST request. In this project, contains resume_text and job_description as JSON fields. FastAPI reads and validates this using a Pydantic model.

Status Code

A 3-digit number in HTTP responses indicating the outcome. 200 = Success, 422 = Validation Error (wrong data), 500 = Server Error.

Uvicorn

An ASGI web server that runs FastAPI applications. Listens for incoming HTTP requests and passes them to FastAPI for processing. Started with uvicorn main:app --reload.

--reload flag

Tells Uvicorn to automatically restart the server whenever a Python file changes. Only used during development, not in production.

Swagger UI (/docs)

FastAPI's automatically generated interactive documentation page. Lists all routes, shows request/response schemas, and allows testing API calls directly in the browser.

127.0.0.1:8000

The local development address. 127.0.0.1 (also called localhost) means this same computer. Port 8000 is where Uvicorn listens by default. Only accessible from your own machine.

Pydantic

Pydantic

A Python library for data validation. When data comes into FastAPI, Pydantic checks that it matches the expected shape and types before the function runs. Invalid data is automatically rejected with a 422 error.

BaseModel

The base class from Pydantic that gives your data class automatic validation superpowers. class ResumeRequest(BaseModel): inherits all of Pydantic's checking capabilities.

Type Hint

A Python annotation that declares what type a variable should be. name: str means name must be text. score: int means score must be a whole number. FastAPI uses these for automatic validation.

ResumeRequest

The custom Pydantic model for this project. Defines that every POST request to /analyze must include resume_text (text) and job_description (text). Missing or wrong-type fields are automatically rejected.

422 Validation Error

The HTTP status code FastAPI/Pydantic returns when incoming data doesn't match the expected schema. If resume_text is missing, a 422 is returned automatically with a clear explanation.

Anthropic Claude API

API (Application Programming Interface)

A set of rules allowing one program to talk to another. An API key is like an ID badge — it proves your app is authorized to use the service.

Anthropic

The AI safety company that created Claude. Mission: build AI systems that are safe, beneficial, and understandable.

Claude
Anthropic's AI assistant/language model. The API version lets your code call Claude programmatically without any human in the loop.

claude-sonnet-4-6
The specific Claude model used in this project. Sonnet is the mid-tier model — balances intelligence, speed, and cost. Better than Haiku, more affordable than Opus.

API Key
A secret credential (long random string starting with sk-ant-api03-) that authenticates your app with Anthropic's servers. Must be kept secret — if exposed, others could use your credit.

.env file
A plain text file that stores secret values like API keys. Never committed to GitHub. Python reads it using load_dotenv(). Format: ANTHROPIC_API_KEY=sk-ant-...

Environment Variable
A variable stored outside your code that your app reads at runtime. Keeps secrets out of source code. Accessed in Python with os.getenv("VARIABLE_NAME").

max_tokens
A parameter that limits how many tokens Claude can use in its response. Set to 2048 in this project to handle longer responses. If exceeded, the response gets cut off.

client.messages.create()
The Python method that sends a message to Claude and gets a response. Takes model, max_tokens, and messages as parameters.

message.content[0].text
How you access Claude's actual text response. content is a list of blocks, [0] gets the first block, .text gets the text string from it.

Prompt Engineering

Prompt Engineering
The practice of writing precise, structured instructions to control an AI model's output format, content, and behavior reliably. Engineering predictable, machine-readable responses.

Prompt
The instruction you send to an AI model. A well-engineered prompt includes: role (who Claude should be), task (what to do), format (how to respond), and constraints (what NOT to do).

Role
Telling Claude who to act as. "You are a resume expert." helps Claude focus its knowledge and tone on the specific domain.

Structured Output
Getting Claude to return data in a specific, parseable format (JSON) instead of free-form prose. Achieved by specifying the exact format and saying "Respond ONLY with..."

JSON

JavaScript Object Notation — a universal text format for structured data exchange between programs. Looks like Python dictionaries: {"key": "value"}. The internet's common language for APIs.

Code Fence

Markdown syntax using triple backticks to format code blocks. Claude sometimes wraps JSON in code fences. Fixed by stripping backticks before parsing with raw.split("```")[1].

Double Curly Braces {{ }}

In Python f-strings, a single {variable} inserts a variable's value. To include a literal { or } character in an f-string, you must double them: {{ and }}. This tells Python it's a real brace, not a placeholder.

Streamlit

Streamlit

A Python library for building web applications without writing HTML, CSS, or JavaScript. You write Python code and Streamlit renders it as an interactive webpage.

st.set_page_config()

Sets the browser tab title and page layout. Must be the first Streamlit command in the file.

st.text_area()

Creates a multi-line text input box. Parameters: label (what shows above the box), height (how tall in pixels).

st.file_uploader()

Creates a file upload button. type=["pdf"] restricts uploads to PDF files only. Returns the uploaded file object or None if nothing uploaded.

st.button()

Creates a clickable button. Everything inside the if st.button("label"): block runs only when the button is clicked.

st.progress()

Displays a progress bar. Takes a value between 0.0 and 1.0. In this project: st.progress(score / 100) converts the 0–100 score.

st.success() / st.info() / st.warning() / st.error()

Colored message boxes: green, blue, yellow, red. Used for color-coded match score feedback.

st.spinner()

Shows a loading animation while code runs inside the with st.spinner(): block.

st.metric()

Displays a large, prominently formatted number with a label. Used for the match score display.

st.divider()

Adds a horizontal line to visually separate sections of the page.

st.caption()

Displays small, muted text. Used for the footer attribution.

requests Library

requests
A Python library for making HTTP requests. Lets your Python code act like a browser — sending GET and POST requests to any URL. Used in Streamlit to call the FastAPI backend.

requests.post()
Sends a POST request to a URL with a JSON body. Parameters: URL string, json= dictionary of data, timeout= maximum seconds to wait. Returns a response object.

response.json()
Parses the JSON response body from an HTTP request into a Python dictionary.

response.status_code
The HTTP status code from the response. Checked to see if the request succeeded (200) or failed (500, 422, etc.).

requests.exceptions.ConnectionError
Raised when the target server is unreachable (FastAPI is not running). Caught with try/except to show a friendly error message.

requests.exceptions.Timeout
Raised when the server takes longer than the timeout limit. Render's free tier can take 30–60 seconds to wake up after inactivity.

timeout=60
Maximum seconds to wait for a response. Set to 60 in the deployed version because Render's free tier sleeps after 15 minutes and can take up to 50 seconds to wake.

pdfplumber

pdfplumber
A Python library for extracting text from PDF files. More reliable than PyPDF2 for text extraction, especially from formatted resumes.

pdfplumber.open(uploaded_file)
Opens a PDF file as a context manager with with to ensure the file is closed properly after use.

pdf.pages
A list of all pages in the opened PDF. A 2-page resume would have pdf.pages = [page1, page2].

page.extract_text()
Extracts all text from a single PDF page as a string. Returns None for blank pages. The if page.extract_text() condition skips blank pages.

"\n".join(...)
Joins all page texts into one large string with a newline between each page, creating the full resume text sent to Claude.

Error Handling

try/except

Python's error handling structure. Code inside try: runs normally. If an error occurs, Python jumps to the matching except: block instead of crashing.

Exception

An error that occurs during program execution. Common: ConnectionError (can't reach server), TimeoutError (server too slow), JSONDecodeError (response isn't valid JSON).

json.JSONDecodeError

Raised when json.loads() can't parse text as valid JSON. Caught in this project to return a helpful error message with the raw response for debugging.

Deployment

Deployment

The process of making an application available on the internet so anyone can access it, not just the developer on their local machine.

Cloud Computer

A physical server sitting in a data center that runs 24/7. Companies like Amazon, Google, and Microsoft rent access to these servers. Your app runs on their hardware, not your laptop.

Render

A cloud platform that hosts web services like FastAPI backends. Free tier available. Services sleep after 15 minutes of inactivity and take ~50 seconds to wake up.

Streamlit Cloud

Cloud platform specifically for hosting Streamlit applications. Free tier available. Reads requirements.txt from the app's folder to install dependencies.

Procfile

A text file that tells Render how to start your application. One line: web: uvicorn backend.main:app --host 0.0.0.0 --port $PORT.

--host 0.0.0.0

Tells Uvicorn to accept connections from any IP address. Required for cloud deployment — without it, only the server itself could access the app.

--port $PORT

Tells Uvicorn which port to listen on, using whatever port Render assigns automatically via the $PORT environment variable.

.gitignore

A file that tells Git which files to never upload to GitHub. The .env file is listed here to ensure your API key never appears in your public repository.

3. Complete Data Flow — Step by Step

Step	What Happens
1	User opens madhu-resume-analyzer.streamlit.app in their browser
2	User uploads a PDF resume or pastes resume text
3	User pastes a job description and clicks Analyze Resume
4	Streamlit extracts text from the PDF using pdfplumber (if PDF uploaded)
5	Streamlit validates that both fields have content
6	requests.post() sends a POST request to https://ml-journey.onrender.com/analyze with resume_text and job_descri
7	Render receives the request and passes it to Uvicorn
8	Uvicorn passes it to FastAPI
9	FastAPI routes it to analyze_resume() function via @app.post("/analyze")
10	Pydantic validates the request data — checks both fields exist and are strings
11	FastAPI builds the engineered prompt with resume and job description inserted
12	client.messages.create() sends the prompt to Claude API on Anthropic's servers
13	Claude (claude-sonnet-4-6) analyzes the texts and generates a JSON response
14	FastAPI receives Claude's response text
15	raw.strip() removes whitespace from the response
16	Code fence stripper removes any ```json ``` wrapping
17	json.loads(raw) converts JSON text to a Python dictionary
18	FastAPI returns the dictionary as JSON response to Streamlit
19	Streamlit receives the JSON and extracts match_score, missing_keywords, suggestion
20	Streamlit displays: progress bar, color-coded score, keyword list, suggestion box

4. Key Code Explained

Loading the API Key

```
load_dotenv(dotenv_path=os.path.join(os.path.dirname(__file__), ".env")) api_key = os.getenv("ANTHROPIC_API_KEY")
```

Finds and loads the .env file from the same folder as main.py. Reads the API key value into a variable. The os.path.dirname(__file__) part gets the directory containing main.py.

Creating the Claude Client

```
client = anthropic.Anthropic(api_key=api_key)
```

Creates a connection to Anthropic's API servers using the key. All future Claude calls use this client object. Think of it as picking up the phone and dialing Claude's number.

Pydantic Data Model

```
class ResumeRequest(BaseModel): resume_text: str job_description: str
```

Defines the shape of incoming POST data. Both fields are required and must be text strings. Pydantic enforces this automatically — no extra validation code needed.

The Analyze Route

```
@app.post("/analyze") def analyze_resume(request: ResumeRequest):
```

Registers a POST route at /analyze. When a POST request hits this URL, FastAPI calls analyze_resume() and passes the validated data as request.

Calling Claude

```
message = client.messages.create( model="claude-sonnet-4-6", max_tokens=2048, messages=[{"role": "user", "content": f"""..."""}] )
```

Sends the engineered prompt to Claude. The f-string inserts request.resume_text and request.job_description into the prompt automatically.

Parsing Claude's Response

```
raw = message.content[0].text raw = raw.strip() if raw.startswith("```"): raw = raw.split("```")[1] if raw.startswith("json"): raw = raw[4:] raw = raw.strip() parsed = json.loads(raw) return parsed
```

Gets Claude's response text, strips whitespace, removes code fences if present, parses JSON into a Python dictionary, returns it as the API response.

Sending Request from Streamlit

```
response = requests.post( "https://ml-journey.onrender.com/analyze", json={"resume_text": resume_text, "job_description": job_description}, timeout=60 )
```

Sends a POST request to the live FastAPI backend on Render with the resume and job description as JSON. Waits up to 60 seconds for a response.

Color-Coded Score Display

```
score = result['match_score'] st.progress(score / 100) if score >= 80:
st.success(f"Strong Match - {score} / 100") elif score >= 60: st.info(f"Good Match -
{score} / 100") elif score >= 40: st.warning(f"Needs Work - {score} / 100") else:
st.error(f"Poor Match - {score} / 100")
```

Gets the match score, shows a progress bar, then shows a color-coded message based on score range. Green (80+), Blue (60-79), Yellow (40-59), Red (0-39).

5. Interview Q&A;

Q: Tell me about Project 4 — what did you build?

I built an AI-Powered Resume Analyzer — a full-stack Python application. The backend is a FastAPI REST API that receives a resume and job description, sends them to the Anthropic Claude API with an engineered prompt, and returns a match score, missing keywords, and improvement suggestion as structured JSON. The frontend is a Streamlit web app where users can upload a PDF or paste text, enter a job description, and instantly see their results. Both are deployed live — FastAPI on Render, Streamlit on Streamlit Cloud.

Q: What is FastAPI and why did you choose it?

FastAPI is a modern Python web framework for building REST APIs. I chose it because it has automatic data validation through Pydantic (which catches bad requests before they reach my business logic), auto-generates interactive documentation at /docs, and is extremely fast. It let me focus on the AI integration logic rather than writing boilerplate validation code.

Q: What is prompt engineering and how did you use it?

Prompt engineering is writing precise instructions to control an AI model's output format and behavior. In this project, I needed Claude to return structured JSON every time so I could parse each field separately in my code. My prompt says: You are a resume expert. Respond ONLY with a JSON object. No markdown, no code fences. Then I specify the exact structure: match_score, missing_keywords, suggestion. I also added a code-fence stripper as a safety net.

Q: How does the frontend talk to the backend?

The Streamlit frontend uses Python's requests library to send an HTTP POST request to the FastAPI backend URL. It passes the resume text and job description as a JSON body. FastAPI receives the request, processes it through Claude, and sends back a JSON response. Streamlit then parses that JSON and displays each field separately on the screen.

Q: How did you handle errors in this project?

I implemented error handling at multiple levels. In Streamlit, I check that both input fields are filled before making the API call. The requests.post() call is wrapped in try/except that catches ConnectionError (FastAPI is down), Timeout (server too slow), and generic Exception for anything unexpected. In FastAPI, I added a code-fence stripper and a json.JSONDecodeError catch that returns a helpful error message with the raw response for debugging.

Q: What is Pydantic and how does it help?

Pydantic is a Python data validation library. I created a ResumeRequest class inheriting from BaseModel. It defines that every POST request to /analyze must have resume_text (string) and job_description (string). If either is missing or wrong type, FastAPI automatically returns a 422 error — I didn't write any validation code myself. This protects my route from receiving garbage data.

Q: How did you deploy this application?

I deployed in two parts. The FastAPI backend runs on Render's free tier — I created a Procfile telling Render to start with uvicorn backend.main:app --host 0.0.0.0 --port $PORT. The API key is

stored as a Render environment variable (not in code). The Streamlit frontend runs on Streamlit Cloud, which reads requirements.txt to install dependencies. Both auto-deploy from GitHub when I push new code.

Q: Why is the API key stored in a .env file?

Security. If I put the API key directly in my Python code, anyone visiting my GitHub repository could see it and use my Anthropic credits. The .env file is listed in .gitignore so Git never uploads it to GitHub. Python reads it using load_dotenv() and os.getenv() accesses the value at runtime. On Render, the key is stored as an environment variable in the dashboard — same concept, different delivery.

6. Day-by-Day Build Log

Day	What Was Built	Key Concept Learned
1	Project structure, venv, FastAPI + Uvicorn installed	Virtual environments, package management
2	First GET route / — Hello World API	Routes, decorators, GET requests, JSON, Uvicorn
3	GET /hello/{name} — time-based greeting	Path parameters, datetime, f-strings, if/elif/else
4	POST /analyze — Pydantic model	POST requests, Pydantic BaseModel, request body, 422 validation
5	Anthropic API connected — first AI response	API keys, .env files, client.messages.create(), Claude integration
6	Prompt engineering — structured JSON output	Prompt engineering, json.loads(), code fence stripping
7	Streamlit frontend built	st.text_area, st.button, requests.post(), full stack integration
8	Error handling added	try/except, ConnectionError, Timeout, input validation
9	PDF upload — pdfplumber integration	pdfplumber, file upload, page.extract_text()
10	UI polish	st.progress, color-coded scores, st.success/info/warning/error
11	End-to-end testing	5 test scenarios, edge case handling
12	Final README writeup	Technical documentation, project presentation
13	Full deployment	Render (backend), Streamlit Cloud (frontend), Procfile, env vars
14	Resume bullet + LinkedIn post	Portfolio presentation, professional communication

7. Resume Bullet & LinkedIn Post

Resume Bullet

```
Built AI-Powered Resume Analyzer — full-stack Python application using FastAPI backend, Anthropic Claude API (prompt engineering for structured JSON output), and Streamlit frontend; deployed live on Render + Streamlit Cloud; analyzes resume-to-job-description match with score (0-100), missing keywords, and improvement suggestions.
```

LinkedIn Post

```
Built something I wish existed when I started job hunting.

An AI-powered Resume Analyzer — upload your resume PDF, paste any job description, get a match score, missing keywords, and one specific suggestion to improve it.

Built with FastAPI + Anthropic Claude API + Streamlit. Deployed live.

Link in comments. What's the #1 thing you wish you knew before applying?

#FastAPI #AI #PromptEngineering #Python #BuildInPublic #DataAnalyst
```

Built from scratch by Madhunika Danam — no templates, no shortcuts. Every line of code written and understood.

danammadhunika@gmail.com · linkedin.com/in/danammadhunika